Cahier des charges

EventSpace : plateforme de réservation de salles événementielles

Projet de classe · Application web full-stack (Angular · Express · EJS · MongoDB · Stripe)

---

1. Présentation du projet

1.1 Contexte

L'organisation d'un événement (mariage, conférence, séminaire, anniversaire, soirée d'entreprise) commence presque toujours par la même difficulté : trouver une salle adaptée, vérifier qu'elle est libre à la bonne date, puis négocier et payer. Aujourd'hui, ce processus repose souvent sur des appels téléphoniques, des échanges de mails dispersés et des plannings tenus à la main, ce qui provoque des erreurs, des doubles réservations et des pertes de temps pour les deux parties.

1.2 Problématique

Comment permettre à un client de trouver, réserver et payer une salle en ligne de façon simple et fiable, tout en donnant au responsable de salle un outil clair pour gérer son offre et valider chaque demande ?

1.3 Solution proposée

EventSpace est une application web qui met en relation deux acteurs :

- le **client**, qui consulte les salles, vérifie leurs disponibilités, envoie une demande de réservation et paie en ligne une fois la demande acceptée ;
- l'**agent de salle**, qui publie et gère ses salles, déclare leurs jours disponibles et valide ou refuse les demandes reçues.

Chaque étape importante déclenche une notification par e-mail : l'agent est prévenu d'une nouvelle demande, le client est informé de la décision.

1.4 Objectifs

Objectifs fonctionnels

- Permettre la réservation d'une salle en ligne, de la recherche jusqu'au paiement.
- Offrir à l'agent un espace de gestion complet de ses salles et de ses réservations.
- Garantir l'absence de double réservation.

- Automatiser les notifications par e-mail.
- Sécuriser le paiement grâce à Stripe.

Objectifs pédagogiques

- Concevoir une architecture avec un backend unique servant deux interfaces de natures différentes (API REST et pages rendues côté serveur).
- Mettre en pratique le travail collaboratif avec Git et GitHub (branches, pull requests, issues).
- Appliquer une démarche agile (user stories, sprints) et produire une documentation de conception (UML).

---

2. Périmètre du projet

2.1 Inclus dans le périmètre (MVP)

- Authentification des deux types d'utilisateurs.
- Gestion des salles, équipements et disponibilités par l'agent.
- Recherche de salles avec filtres et consultation du calendrier par le client.
- Demande de réservation avec nombre d'heures.
- Validation ou refus par l'agent, avec motif en cas de refus.
- Paiement en ligne via Stripe (mode test).
- Notifications par e-mail.
- Suivi et annulation des réservations.

2.2 Hors périmètre de la première version

- Application mobile native.
- Paiement en conditions réelles (le projet reste en mode test).
- Messagerie instantanée entre client et agent.
- Gestion de plusieurs devises ou langues.
- Remboursement automatique intégral (traité comme évolution).

---

3. Acteurs du système

Acteur	Rôle UML	Description	Interface
Client	Principal	Utilisateur pour qui l'application existe. Il déclenche le but central : réserver une salle et payer.	Angular (SPA), authentification JWT
Agent de salle	Secondaire	Responsable d'une ou plusieurs salles. Il prépare l'offre et valide ou refuse les demandes.	EJS (pages serveur), authentification par session
Stripe	Système externe	Service de paiement en ligne.	API Stripe Checkout et webhooks
Service e-mail	Système externe	Envoi des notifications.	SMTP via Nodemailer

---

4. Exigences fonctionnelles

Les priorités suivent la méthode MoSCoW : **M** (Must, indispensable), **S** (Should, important), **C** (Could, bonus).

4.1 Module Authentification

Réf.	Exigence	Acteur	Priorité
F-01	Le client peut créer un compte (nom, e-mail, mot de passe).	Client	M
F-02	Le client peut se connecter et se déconnecter.	Client	M
F-03	L'agent peut se connecter et se déconnecter.	Agent	M
F-04	Les mots de passe sont stockés hachés (bcrypt).	Système	M
F-05	L'accès aux fonctionnalités est restreint selon le rôle.	Système	M
F-06	Réinitialisation du mot de passe par e-mail.	Client, Agent	C

4.2 Module Gestion des salles (agent)

Réf.	Exigence	Priorité
F-10	Créer une salle : nom, description, capacité, prix par heure, adresse, photos.	M
F-11	Modifier et supprimer une salle dont il est propriétaire.	M
F-12	Associer des équipements à une salle (projecteur, sonorisation, climatisation, Wi-Fi, scène, parking, etc.).	M
F-13	Consulter la liste de toutes ses salles.	M
F-14	Téléverser plusieurs photos par salle.	S
F-15	Activer ou désactiver temporairement la publication d'une salle.	C

4.3 Module Disponibilités (agent)

Réf.	Exigence	Priorité
F-20	Déclarer des dates précises où la salle est disponible.	M
F-21	Déclarer des jours récurrents (par exemple tous les lundis, mardis et vendredis), convertis en dates pour les semaines à venir.	M
F-22	Bloquer une date (maintenance, fermeture, événement privé).	M
F-23	Supprimer ou modifier une disponibilité qui n'a pas de réservation validée.	M
F-24	Visualiser le planning d'une salle sous forme de calendrier.	S

4.4 Module Recherche et consultation (client)

Réf.	Exigence	Priorité
F-30	Consulter la liste des salles publiées.	M

Réf.	Exigence	Priorité
F-31	Filtrer par date, capacité minimale, prix maximum et équipements.	M
F-32	Consulter la fiche détaillée d'une salle (photos, équipements, prix, adresse).	M
F-33	Consulter le calendrier des jours disponibles d'une salle.	M
F-34	Ajouter une salle aux favoris.	C
F-35	Consulter et déposer des avis et des notes.	C

4.5 Module Réservation

Réf.	Exigence	Acteur	Priorité
F-40	Envoyer une demande : salle, jour disponible, nombre d'heures, nombre d'invités, type d'événement.	Client	M
F-41	Le système calcule et affiche en direct le montant total (prix par heure × nombre d'heures).	Système	M
F-42	Consulter la liste de ses réservations avec leur statut.	Client	M
F-43	Annuler une réservation dans le délai autorisé.	Client	M
F-44	Consulter les demandes reçues, filtrables par statut, salle et date.	Agent	M
F-45	Valider une demande.	Agent	M
F-46	Refuser une demande en indiquant un motif.	Agent	M
F-47	Expiration automatique d'une réservation validée mais non payée après 48 heures.	Système	S
F-48	Consulter l'historique du client (réservations passées sur ses salles, statuts, montants, indicateurs) avant de valider ou de refuser une demande.	Agent	M

4.6 Module Paiement (Stripe)

Réf.	Exigence	Priorité
F-50	Le client paie une réservation validée via Stripe Checkout.	M
F-51	Le statut passe à « Payée » uniquement à la réception du webhook Stripe `checkout.session.completed`.	M
F-52	Le client peut consulter l'historique de ses paiements.	S
F-53	Génération d'un reçu ou d'une facture au format PDF.	C

4.7 Module Notifications par e-mail

Réf.	Événement déclencheur	Destinataire	Priorité
F-60	Nouvelle demande de réservation	Agent	M
F-61	Réservation validée (avec lien de paiement)	Client	M
F-62	Réservation refusée (avec motif)	Client	M
F-63	Paiement confirmé	Client et agent	S
F-64	Réservation annulée ou expirée	Client et agent	S

4.8 Module Tableau de bord (agent)

Réf.	Exigence	Priorité
F-70	Afficher le nombre de demandes en attente, validées et payées.	S
F-71	Afficher les revenus par mois et par salle.	C

5. Cycle de vie d'une réservation

Statut	Signification	Transitions possibles
En attente	Demande envoyée par le client, en attente de décision.	Validée, Refusée, Annulée
Validée	Acceptée par l'agent, en attente de paiement.	Payée, Expirée, Annulée
Refusée	Rejetée par l'agent avec un motif.	(état final)
Payée	Paiement Stripe confirmé par webhook.	Terminée, Annulée
Expirée	Non payée dans les 48 h suivant la validation.	(état final)
Annulée	Annulée par le client ou l'agent.	(état final)
Terminée	La date de l'événement est passée.	(état final)

Déroulement nominal : demande du client → e-mail à l'agent → validation → e-mail au client avec lien de paiement → paiement Stripe → webhook → réservation payée.

---

6. Règles de gestion

Réf.	Règle
RG-01	Un agent peut posséder plusieurs salles. Chaque salle appartient à un seul agent.
RG-02	Un agent ne peut consulter et gérer que les salles et réservations qui lui appartiennent.
RG-03	Un client ne peut réserver que sur une date marquée comme disponible par l'agent.
RG-04	Il est impossible de réserver une date passée ou une date bloquée.
RG-05	Le nombre d'invités ne doit pas dépasser la capacité de la salle.
RG-06	Le nombre d'heures doit être strictement positif et ne pas dépasser la durée maximale d'ouverture définie pour la salle.

Réf.	Règle
RG-07	Une même salle ne peut avoir qu'une seule réservation validée ou payée pour un même jour (option A retenue ; une évolution vers un chevauchement horaire est possible).
RG-08	Montant = prix par heure × nombre d'heures.
RG-09	Un refus doit obligatoirement être accompagné d'un motif.
RG-10	Une réservation validée et non payée expire après 48 heures.
RG-11	Une annulation est possible jusqu'à un nombre de jours défini avant l'événement (par exemple 7 jours).
RG-12	Le paiement n'est possible que pour une réservation au statut « Validée ».
RG-13	Une réservation payée ne peut être modifiée que par annulation.
RG-14	Le prix par heure, la capacité, les équipements et les photos d'une salle sont définis par l'agent. Le client peut seulement les consulter.
RG-15	Le prix affiché sur une salle est le prix d'une heure. Le total est recalculé en direct quand le client change le nombre d'heures (exemple : 30 € × 10 h = 300 €).
RG-16	L'historique d'un client regroupe ses réservations passées sur les salles de l'agent connecté (date, salle, montant, statut) et des indicateurs : nombre de réservations, payées, annulées, refusées et expirées.

7. Exigences non fonctionnelles

7.1 Sécurité

- Mots de passe hachés avec bcrypt, jamais stockés en clair.

- Authentification par JWT pour l'API et par session sécurisée (cookie `httpOnly`) pour l'interface agent.
- Contrôle d'accès par rôle et vérification de propriété sur chaque ressource.
- Validation et nettoyage de toutes les entrées côté serveur (`express-validator`).
- Protection CSRF sur les formulaires EJS.
- Les clés Stripe, identifiants SMTP et secrets sont dans des variables d'environnement, jamais dans le dépôt.
- Vérification de la signature de chaque webhook Stripe.

7.2 Performance et fiabilité

- Temps de réponse de l'API inférieur à 2 secondes en usage normal.
- Index MongoDB sur les champs de recherche et index unique empêchant physiquement les doubles réservations.
- Gestion centralisée des erreurs avec messages clairs pour l'utilisateur.

7.3 Ergonomie et compatibilité

- Interface responsive (ordinateur, tablette, mobile).
- Parcours de réservation en trois étapes maximum après le choix de la salle.
- Compatibilité avec les versions récentes de Chrome, Firefox, Edge et Safari.

7.4 Maintenabilité

- Logique métier centralisée dans des services réutilisés par l'API et par les routes EJS.
- Code organisé par couches (routes, contrôleurs, services, modèles).
- Conventions de nommage et de commits communes à l'équipe.

---

8. Architecture technique

8.1 Vue d'ensemble

Couche	Technologie	Rôle
Interface client	Angular	Application monopage consommant l'API REST
Interface agent	EJS	Pages HTML générées côté serveur
Backend	Node.js et Express	API REST, pages EJS, logique métier

---

Couche	Technologie	Rôle
Base de données	MongoDB et Mongoose	Persistance des données
Paiement	Stripe Checkout et webhooks	Paiement en ligne sécurisé
E-mails	Nodemailer (Mailtrap en développement)	Notifications
Versionnement	Git et GitHub	Travail collaboratif

8.2 Principe

Un **unique backend Express** sert deux interfaces de natures différentes. Le client Angular dialogue avec une API REST en JSON et s'authentifie par JWT. L'agent utilise des pages EJS rendues par le serveur et s'authentifie par session stockée dans MongoDB (`connect-mongo`). La logique métier est écrite une seule fois dans des services partagés (réservation, disponibilité, paiement, e-mail).

8.3 Structure du backend

```
backend/
├── app.js
├── config/        (base de données, variables d'environnement)
├── models/        (User, Room, Availability, Booking, Payment)
├── services/      (bookingService, availabilityService, paymentService,
emailService)
├── middlewares/   (authJwt, authSession, roleCheck, errorHandler)
├── routes/
|   ├── api/       (pour Angular)
|   └── agent/     (pour EJS)
├── views/         (pages EJS)
└── public/        (css, js, uploads)
```

---

9. Modèle de données (aperçu)

Entité	Attributs principaux
User	nom, e-mail, mot de passe haché, rôle (client ou agent), date de création

Entité	Attributs principaux
Room	nom, description, capacité, prix par heure, adresse, photos, équipements, publiée, agent (réf.)
Availability	salle (réf.), date, statut (ouverte, bloquée, réservée)
Booking	client (réf.), salle (réf.), date, nombre d'heures, nombre d'invités, type d'événement, montant, statut, date d'expiration, motif de refus
Payment	réservation (réf.), montant, identifiant de session Stripe, statut, date de paiement

Le diagramme de classes détaillé fait l'objet d'un livrable séparé.

---

10. Organisation du travail

10.1 Méthode

Le projet suit une démarche **agile** : les besoins sont exprimés en user stories, regroupées dans un product backlog puis planifiées en sprints courts, avec une démonstration à la fin de chaque sprint.

10.2 Découpage indicatif des sprints

Sprint	Objectif	Contenu principal
Sprint 0	Fondations	Dépôt GitHub, squelette Express, connexion MongoDB, modèles, projet Angular
Sprint 1	Espace agent	Authentification, CRUD des salles, équipements, disponibilités
Sprint 2	Cœur de la réservation	API de recherche, création de demande, validation et refus, e-mails
Sprint 3	Paiement et interface client	Stripe Checkout et webhook, pages Angular, suivi et annulation
Sprint 4	Finalisation	Tests, corrections, bonus, documentation et présentation

10.3 Répartition des rôles (proposition)

Domaine	Responsabilités
Interface agent (EJS)	Pages, formulaires, gestion des salles et des disponibilités
API et logique métier	Modèles, services, règles de gestion, authentification
Interface client (Angular)	Pages, appels API, calendrier, parcours de réservation
Paiement, e-mails et qualité	Stripe, Nodemailer, tests, documentation

10.4 Workflow GitHub

- Branches : `main` (stable), `develop` (intégration), `feature/nom-de-la-fonctionnalité`.
- Chaque tâche correspond à une issue GitHub et à une branche dédiée.
- Fusion uniquement par pull request, avec relecture d'au moins un coéquipier.
- Suivi via un GitHub Project (À faire, En cours, En revue, Terminé).
- Fichier `.gitignore` excluant `node_modules` et `.env`, et fichier `.env.example` commité.
- Messages de commit clairs, par exemple `feat: ajout du filtre par capacité`.

---

11. Livrables

N°	Livrable	Format
1	Cahier des charges	Document
2	Product backlog et sprint backlog (user stories)	Tableau
3	Diagramme de cas d'utilisation	UML
4	Diagramme de classes	UML
5	Maquettes et charte graphique	HTML ou images
6	Application fonctionnelle	Dépôt GitHub
7	Documentation technique et guide d'installation	README

N°	Livrable	Format
8	Présentation et démonstration finale	Diaporama et démo

12. Analyse des risques

Risque	Probabilité	Impact	Mesure préventive
Double réservation d'une même salle	Moyenne	Élevé	Index unique MongoDB et vérification dans le service
Conflits de fusion Git	Élevée	Moyen	Branches par fonctionnalité, pull requests fréquentes
Webhook Stripe difficile à tester en local	Moyenne	Moyen	Stripe CLI dès le début du sprint 3
E-mails non reçus pendant la démonstration	Moyenne	Moyen	Mailtrap en développement, compte SMTP de secours
Retard sur le périmètre	Moyenne	Élevé	MVP prioritaire, bonus uniquement en fin de projet
Deux authentifications différentes (JWT et session) mal séparées	Moyenne	Moyen	Middlewares distincts et tests par rôle

13. Critères d'acceptation globaux

Le projet est considéré comme réussi lorsque :

1. Un client peut s'inscrire, trouver une salle, demander une réservation, être notifié de la décision et payer via Stripe en mode test.
2. Un agent peut gérer plusieurs salles, leurs équipements et leurs disponibilités, et valider ou refuser une demande.
3. Aucune double réservation n'est possible, même en cas de demandes simultanées.
4. Les e-mails prévus sont envoyés aux bons destinataires.
5. Les deux interfaces sont fonctionnelles, cohérentes et utilisables sur mobile.
6. Le code est versionné sur GitHub avec un historique de contributions de chaque membre.

14. Évolutions possibles

- Réservation par créneau horaire avec détection des chevauchements.
- Tarification variable selon la saison ou le jour.
- Avis et notes, favoris.
- Facture PDF téléchargeable.
- Remboursement automatique via l'API Stripe.
- Tableau de bord statistique avancé.
- Notifications en temps réel.